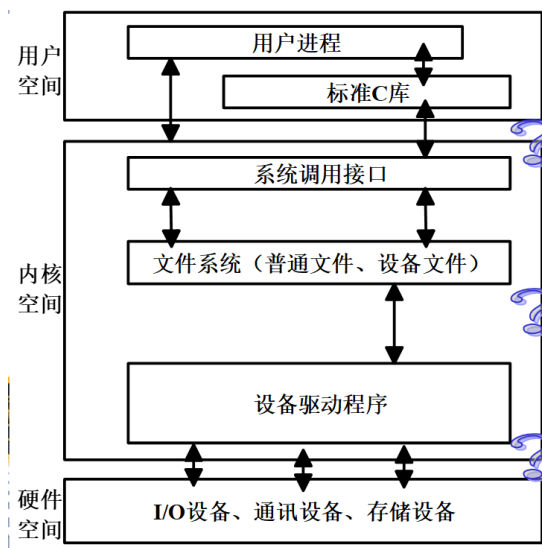


第 04 讲 嵌入式系统设备驱动程序 1

一、 设备驱动程序简介

为什么需要设备驱动程序：

设备驱动程序如何工作：



➤ 设备驱动程序简介

设备驱动程序 (Device Driver) 简称“驱动程序”，是一种可以使计算机和设备通信的特殊程序，可以说相当于硬件的接口。操作系统通过这个接口，才能控制硬件设备的工作。

系统调用是操作系统内核和应用程序之间的接口

设备驱动程序是操作系统内核和机器硬件之间的接口

设备驱动程序作为内核模块动态加载

设备驱动程序为应用程序屏蔽了硬件的细节

在应用程序看来，硬件设备只是一个设备文件，应用程序可以象操作普通文件一样对硬件设备进行操作。

从用户角度出发，用户希望能使用同样的应用程序接口和命令来访问设备和普通文件。

- Linux 抽象了对硬件的处理, 所有的硬件设备都可以作为普通文件来看待; 它们可以使用和操作文件相同的、标准的系统调用接口来完成打开、关闭、读写和 I/O 控制操作，而驱动程序的主要任务也就是要实现这些系统调用。

设备即文件

➤ 设备驱动程序完成以下的功能:

- ☐ 对设备初始化和释放;
- ☐ 把数据从内核传送到硬件和从硬件读取数据;
- ☐ 读取应用程序传送给设备文件的数据和回送应用程序请求的数据;
- ☐ 检测和处理设备出现的错误;

➤ Linux 操作系统的设备分类:

- ☐ 字符设备
- ☐ 块设备

❑ 网络设备

➤ 字符设备

字符设备以单个字节为单位进行顺序读写操作，通常不使用缓冲技术。

典型的字符设备包括鼠标、键盘、串行口等。

字符设备驱动通常至少实现 open、close、read 和 write 等系统调用。

与普通文件的区别主要在于大部分字符设备只能顺序访问数据通道。

➤ 块设备

块设备是以固定大小的数据块进行存储和读写的设备，例如硬盘、软盘、CD-ROM 等。

为提高效率，对于块设备，系统利用一块系统内存作为缓冲区，为块设备的读写提供了缓存机制，由于涉及缓冲区管理、调度和同步等问题，块设备实现起来比字符设备复杂得多。

块驱动程序，除了给内核提供和字符驱动一样的接口外，还提供了专门面向块设备的接口，块设备的接口必须支持挂载文件系统。

➤ 网络设备

网络设备在 Linux 操作系统里进行了专门的处理。

Linux 的网络系统主要是基于 BSD UNIX 的 Socket 机制。

在系统和驱动程序之间有专门的数据结构（sk_buff）进行数据的传递，负责发送和接收数据包。

系统支持对发送数据和接收数据的缓存，提供流量控制机制，提供对多协议的支持。

二、Linux 设备驱动程序原理

用户进程运行在用户空间，设备驱动程序工作在内核空间

系统调用是用户进程进入内核空间的唯一通道。

每当用户进程使用系统调用时，都自动地将运行模式从用户级转为内核级，此时进程在内核的地址空间中运行。

Linux 抽象了对硬件的处理，各种设备都以文件的形式存放在/dev 目录下，称为设备文件。

所有的硬件设备都可以作为普通文件来看待，应用程序可以打开、关闭和读写这些设备文件，完成对设备的操作，就像操作普通的数据文件一样

➤ Linux 的设备驱动程序源代码都放在 Linux/drivers 目录下，主要包括：

- ❑ /block 目录：块设备的驱动程序；
- ❑ /char 目录：字符设备的驱动程序；
- ❑ /cdrom 目录：CD-ROM 代码的驱动；
- ❑ /pci 目录：PCI 设备的驱动代码；
- ❑ /scsi 目录：所有 SCSI 设备的驱动；
- ❑ /net 目录：网络驱动代码；
- ❑ /sound 目录：所有的声卡驱动代码。

➤ 设备号

嵌入式 Linux 系统通过设备号来区分不同设备

➤ 设备号分为

❑ 主设备号

- 内核通过主设备号将设备与相应的驱动程序对应起来
- 主设备号的取值范围是 0 ~ 255

❑ 次设备号

- 当一个驱动程序要控制若干个设备时，就要用次设备号来区分它们。

➤ 对自定义的产品设备，用户需定义设备的主设备号，注意，自定义设备的主设备号不能与已存在的主

设备号冲突

- 通过位于文件系统/dev 目录下的设备文件，可查看每个设备的名称、主从设备号及文件属性等信息

eg. 举例：

```
ls -l /dev/ttyS2
```

输出：

```
crw-rw- - - root 4,67 2003-01-30 /dev/ttyS2
```

说明：

4, 67 表示主设备号为 4，次设备号为 67

Linux 常见设备文件：

/dev/ttyS0：第一个串行口。

/dev/ttyS1：第二个串行口。

/dev/ttyS2 和/dev/ttyS3 是第三和第四个串行口。

/dev/modem：串口调制解调器。

/dev/mouse：鼠标，通常是一个指向/dev/ttyS0 或相似设备的符号链接，具体取决于鼠标接在哪个串行端口。

/dev/lp0：接在第一个并行口的打印机。

/dev/lp1：接在第二个并行口的打印机。

/dev/fd0：第一个软盘驱动器。

/dev/fd0H1440：第一个软盘驱动器的高密度模式驱动程序。

/dev/fd1：第二个软盘驱动器。

/dev/had：第一个 IDE 硬盘。

/dev/hdb：第二个 IDE 硬盘。

/dev/hdc：第三个 IDE 硬盘。

/dev/cdrom：指向 CDROM。

/dev/hda1：第一个 IDE 硬盘的第一个分区。/dev/hda2 是第一个 IDE 硬盘的第二个分区。

/dev/tty1：第一个字符终端。/dev/tty2 是第二个字符终端，以此类推。

/dev/dsp：数字音频，例如声卡。

/dev/sda：第一个 SCSI 硬盘。

/dev/sdb：第二个 SCSI 硬盘。

/dev/sda1：第一个 SCSI 硬盘的第一个分区。

/dev/sr0：第一个 SCSI 光驱。

/dev/usb/scanner0：一个 usb 扫描仪。

➤ 设备注册

- 设备获得主、次设备号有两种方式：

手动给定一个数，使用某个函数注册，并将它与设备联系起来；

调用系统函数给设备动态分配一个主、次设备号。

注意：不管是哪种方法获取设备号，都需要在/dev 目录下用 mknod 命令建立相应设备的设备标识。

手动给定设备号：

字符设备的注册函数：

```
register_chrdev(unsigned int major, const char * name, struct file_operations *fops);
```

其中

major 是设备驱动程序向系统申请的主设备号

name 是设备名，即驱动程序的名称

fops 是驱动程序中所定义的结构体的地址

从本质上来说，设备注册的过程，其实就是将设备驱动程序与该设备的设备号及设备名(设备进入点)相关联。

举例：

```
register_chrdev(Demo_MAJOR, "demo_drv", &Demo_ops);
```

动态分配主设备号

```
int alloc_chrdev_region(dev_t*dev,unsigned int firstminor, unsigned int count, char* name);
```

其中

dev 是一个只输出的参数，它在函数成功完成时持有设备分配范围的第一个数；

firstminor 是请求的第一个要用的次设备号，通常是 0

count 是所请求的连续设备号的个数

name 代表驱动程序名称

字符设备的注销函数：

```
unregister_chrdev (unsigned int major, const char * name );
```

其中

major 和 name 必须与传递给 register_chrdev 函数的值保持一致，否则该调用会失败。

举例

```
unregister_chrdev (Demo_MAJOR, "demo_drv" );
```

➤ 驱动调用接口函数

设备驱动程序是一组由内核中的相关子例程和数据组成的 I/O 设备软件接口。

在 Linux 系统中字符设备驱动器保存在 /usr/src/linux/drivers/char 目录中。

Linux 为所有的设备文件都提供了统一的操作函数接口，方法是

使用数据结构 struct file_operations。

struct file_operations 结构体中的成员为一系列的接口函数。

➤ file_operations 结构：

```
struct file_operations {  
    struct module *owner;  
    loff_t (*llseek) (struct file *, loff_t, int);  
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);  
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);  
    unsigned int (*poll) (struct file *, struct poll_table_struct *);  
    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);  
    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);  
    int (*mmap) (struct file *, struct vm_area_struct *);  
    int (*open) (struct inode *, struct file *);  
    int (*release) (struct inode *, struct file *);  
    int (*flush) (struct file *, fl_owner_t id);  
    int (*fsync) (struct file *, loff_t, loff_t, int datasync);  
    ... ..};
```

这个结构的每一个成员的名字都对应着一个系统调用。

Linux 的设备驱动程序工作的基本原理

用户进程利用系统调用在对设备文件进行操作时，系统调用通过设备文件的主设备号找到相应的设备驱动程序，然后读取这个数据结构相应的函数指针，接着把控制权交给该函数

编写设备驱动程序的主要工作就是编写子函数，并填充 file_operations 的各个域。

file_operations 结构举例：

```
#include <linux/fs.h>
```

```
struct file_operations GPIO_LED_ctl_ops = {
```

```

open:      SIMPLE_GPIO_LED_open,
read:      SIMPLE_GPIO_LED_read,
write:     SIMPLE_GPIO_LED_write,
unlocked_ioctl: SIMPLE_GPIO_LED_ioctl,
release:   SIMPLE_GPIO_LED_release,};

```

➤ file 结构:

该结构代表一个打开的文件，内核在 open 时创建，并传递给在该文件上进行操作的所有函数。它提供关于被打开的文件的信息。

```

struct file{
const struct file_operations * f_op; //与文件相关的操作
struct path    f_path;
fmode_t      f_mode; //文件模式，表示文件是否可读、可写
off_tf_pos; //当前的文件读写位置
unsigned int   f_flags; //文件标志
atomic_long_tf_count;
struct fown_struct f_owner;
void*         private_data;
... ...};

```

➤ inode 结构

内核用 inode 结构在内部表示文件。

dev_t i_rdev

包含了真正的设备编号

struct cdev_t *i_rdev

表示字符设备的内核的内部结构，当 inode 指向一个字符设备文件时，该字段包含了指向 struct cdev 结构的指针

```

struct cdev {
    struct kobject kobj;           //内嵌的内核对象.
    struct module *owner;          //该字符设备所在的内核模块的对象指针.
    const struct file_operations *ops; //与文件相关的操作，是极为关键的一个结构体.
    struct list_head list;         //用来将已经向内核注册的所有设备形成链表.
    dev_t dev;                     //设备的设备号，由主设备号和次设备号构成.
    unsigned int count;            //隶属于同一主设备号的次设备号的个数.
};

```

➤ (1) open 函数

在设备驱动程序中，设备的打开操作由功能接口函数 open() 完成。它主要提供驱动程序初始化的能力，为以后对设备进行 I/O 操作做准备。

```
int (*open)(struct inode*, struct file*);
```

其中：

inode 是设备文件的 inode 结构的指针

file 是指向这一设备文件结构的指针

返回值为 0，表示设备打开；返回值为负数，打开失败

需要的头文件为 linux/kernel.h

open 函数的工作:

检查设备相关错误。
如果是第一次打开, 则初始化硬件设备。
识别设备号。
分配和填写要放在 file->private_data 里的数据结构
使用计数增 1

➤ (2) release 函数

release() 函数是释放设备的接口。

int (*release)(struct inode*, struct file *)

需要的头文件为 linux/kernel.h

工作:

释放由 open 分配的, 保存在 file->private_data 里的所有内容
在最后一次关闭操作时关闭设备
使用计数减 1

➤ (3) 设备的读写操作

ssize_t read(struct file *filp, char *buff, size_t count, loff_t *offp)

filp 是文件指针

buff 为存放读取结果的缓冲区

count 为所要读取的数据长度

offp 是一个指向“long offset type”对象的指针, 该对象指明用户在文件中进行存取操作的位置。

作用: 将数据从设备复制数据到用户空间

返回值为负, 表示读取操作发生错误, 否则, 返回实际读取的字节数。

ssize_t write(struct file *filp, const char *buff, size_t count, loff_t *offp);

作用: 将数据从用户空间复制到设备上

返回值非负, 表示成功写入的字节数

read() 和 write() 需要的头文件为 linux/fs.h

注意: 这两个函数需要用到一个 **buffer** 参数, 该参数指向用户空间的缓冲区, 而内核代码不能直接使用, 因此需要使用如下两个函数实现数据的复制。

copy_to_user (): 从内核空间传送数据给 buffer

copy_from_user (): 将 buffer 中的数据复制到内核

➤ (4) 设备的控制操作

在设备驱动程序中, 接口函数 ioctl() 主要用于对设备进行读写之外的其他控制操作。

ioctl () 函数的定义:

int (*ioctl) (struct file *filp, unsigned int cmd, unsigned long arg);

其中

cmd 是用户程序对设备的控制命令

arg 表示可选参数, 有或没有是和 cmd 的意义相关的。

接口函数 ioctl() 所需的头文件为: linux/fs.h

➤ 系统调用与用户程序

用户通过调用系统提供的**系统调用函数**来实现对设备驱动程序的使用和操作。

(1) 文件描述符

对于 Linux 而言，所有**对设备和文件的操作**都是使用**文件描述符**来进行的。

文件描述符是一个非负的整数，**它是一个索引值**，并指向内核中每个进程打开文件的记录表。

当打开一个现存文件或创建一个新文件时，内核就向进程返回一个文件描述符；当需要读/写文件时，也需要把文件描述符作为参数传递给相应的函数。

通常，一个进程启动时，都会打开 3 个文件：标准输入、标准输出和标准出错处理。这 3 个文件分别对应的文件描述符为 0, 1, 2

查看 Linux 默认的文件描述符命令

```
ulimit -n
```

(2) 使用文件描述符的系统调用

➤ open 调用

功能：

打开一个文件，并指定访问该文件的方式，调用成功后返回一个**文件描述符**，发生错误时返回-1。

原型：

```
int open(const char * pathname, int flags);
```

其中：

pathname 是要打开文件的完全路径名或相对路径名，是一个字符串指针。

flags 可以是 O_RDONLY（只读）、O_WRONLY（只写）和 O_RDWR（读写）3 个值之一。

举例：

```
int fd = open(DEVICE_NAME,O_RDWR);
```

➤ close 调用

功能：

使用完文件后调用 close 关闭相应的文件描述符

原型：

```
int close(int fd);
```

其中

fd 是文件描述符，该文件描述符是在调用 open 时返回的。

举例：

```
close(fd);
```

➤ read 调用

功能：

从文件描述符对应的文件中读取数据，调用成功后返回读出的字节数，失败时返回-1。

原型：

```
ssize_t read(int fd, void* buf, size_t count );
```

其中

fd 是从 open 调用返回的文件描述符。

buf 是缓冲区的指针，读出的数据将被存放到该缓冲区中。

count 要读取的字节数

举例：

```
read(fd,NULL,0);
```

➤ write 调用

功能：

向文件描述符对应的文件中写入数据，调用成功后返回写入的字节数，失败时返回-1。

原型：

```
ssize_t write(int fd, void* buf, size_t count );
```

其中

fd 是从 open 调用返回的文件描述符。

buf 是缓冲区的指针，用于存放要写入文件的数据。其大小必须满足能够放得下这些数据的要求

count 要写入的字节数

举例：

```
write(fd,NULL,0);
```

➤ ioctl 调用

功能：

设置或检索文件的有关参数并对文件进行一些其他的操作，涉及的设备不同，其参数也不同

原型：

```
int ioctl(int fd, int cmd, long arg );
```

```
int ioctl(int fd, int cmd);
```

其中

fd 是从 open 调用返回的文件描述符。

cmd 是命令

arg 要特殊的命令参数

举例：

```
ioctl(fd, LED_ON);
```

➤ lseek 调用

功能：

在文件描述符对应的文件里把文件指针设定到指定的位置，调用成功后返回新指针的位置 offset。失败时返回-1。

原型：

```
off_t lseek(int fildes, off_t offset, int whence );
```

其中

offset 所指的偏移位置与 whence 有关， whence 有 3 个取值：

SEEK_SET：从文件的开始处计算偏移。

SEEK_CUR：从当前的文件偏移值计算偏移。

SEEK_END：从文件的结束处计算偏移。

三、 设备驱动程序和用户应用程序

➤ 对设备进行访问和操作的程序由两部分组成

- 设备驱动程序
- 用户应用程序

➤ 应用程序/驱动程序区别：

1.应用程序一般有一个 main 函数，从头到尾执行一个任务；

驱动程序却不同，它没有 main 函数，通过使用宏 module_init(初始化函数名)，将初始化函数加入内核全局初始化函数列表中。通过宏 module_exit(退出处理函数名)注册退出处理函数

2.应用程序可以包含标准的头文件，比如<stdio.h>、<stdlib.h>等；

在驱动程序中是不能使用标准 C 库的，因此不能调用所有的 C 库函数，只能调用内核的函数，比如输出打印函数只能使用内核的 `printk` 函数，包含的头文件只能是内核的头文件 `<linux/kernel.h>`

➤ `printk` 函数

■ 功能：打印信息

■ 说明：

- ◆ 与 `printf` 函数类似，都可以按照一定的格式打印信息
- ◆ 与 `printf` 函数不同，可以定义打印信息的优先级
- ◆ 必须启动 `klogd` 和 `syslogd` 服务，信息才能输出
- ◆ 信息默认输出到系统日志文件（例如：“`/var/log/messages`”）
- ◆ 可以输出到虚拟控制台上，对输出给控制台的信息有一个特定的优先级 `console_loglevel`。打印信息的优先级小于这个值，信息才能被输出到虚拟控制台上，否则信息只写入系统日志文件中。

➤ 设备驱动程序的框架

例 1：一个最简单的驱动程序（只有加载和卸载功能）

```
#include <linux/module.h>
```

```
#include <linux/kernel.h> //驱动程序必不可少的
```

```
int init_module(void)
```

```
{
```

```
    printk("Hello,Test_drv [ ---kernel---]\n");在控制台上显示输出，包含在 kernel.h 中
```

```
    return 0;
```

```
}
```

```
void cleanup_module(void)
```

```
{
```

```
    printk("Goodbye Test_drv [ ---kernel---]\n");//在控制台上显示输出，包含在 kernel.h 中
```

```
}
```

```
module_init(init_module);
```

```
//加载驱动的入口点
```

```
module_exit(cleanup_module);
```

```
//卸载驱动的入口点
```

```
#include <...../xxx.h> //驱动程序所必须的包含文件
```

```
open(){...}
```

```
read(){...}
```

```
write(){...}
```

```
.....
```

```
struct file_operation{
```

```
    .....
```

```
};
```

// 设备的功能接口函数与数据结构体

```
int init_module(void)
```

```
{ ..... //驱动程序注册语句 }
```

```
void cleanup_module(void)
```

```
{ ..... //释放设备资源语句 }
```

```
module_init(init_module); //加载驱动的入口点
```

```
module_exit(cleanup_module); //卸载设备驱动的入口点
```

四、设备驱动程序示例

➤ 设备进入点

对每个设备都要定义一个设备进入点，该设备进入点的名称则称为设备名。设备进入点又称为设备文件。

如果设备注册成功，则设备名就会写入到/proc/devices 文件中。

(1) 创建设备进入点——mknod

(2) 查看设备进入点——ls

(3) 删除设备进入点——rm

(1) 创建设备进入点

创建设备进入点的命令格式为：

mknod /dev/xxx type major minor

其中：

xxx 为设备名；

type 为设备类型，若为字符设备，则为 c，若为块设备，则为 b；

major 和 minor 分别为主设备号、次设备号。

举例

```
[root@BC root]# mknod /dev/demo_drv c 98 0
```

(2) 查看设备进入点

查看设备进入点是否创建成功，命令的一般格式为：

```
ls -l /dev |grep 设备名
```

举例

```
[root@BC root]# ls -l /dev |grep demo_drv
```

(3) 删除设备进入点

```
[root@BC root]# rm /dev/demo_drv
```

➤ 加载设备驱动程序

(1) 加载设备驱动程序

insmod < 设备驱动程序.ko >

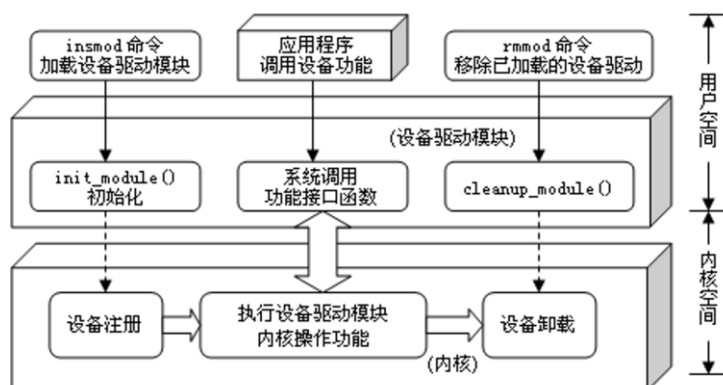
(2) 查看当前加载的设备驱动程序

lsmod

(3) 卸载驱动程序

rmmod < 设备驱动程序 >

➤ 设备驱动程序加载与卸载的工作过程



➤ 设备驱动程序的功能接口函数模块

设备驱动程序通过功能接口函数完成对设备的各种操作。

一个设备驱动程序模块包含有 5 个部分的功能接口函数：

- (1) 驱动程序的注册与释放 (register_chrdev、unregister_chrdev)
- (2) 设备的打开与关闭 (open、release)
- (3) 设备的读写操作 (read、write)
- (4) 设备的控制操作 (ioctl)
- (5) 设备的中断与轮询处理

➤ 设备驱动程序重要的数据结构体

用户应用程序调用设备的功能都是在设备驱动程序中定义的，也就是设备驱动程序中所定义的功能入口点函数（或称为功能接口函数）。这些设备的功能接口函数都被定义在 <linux/fs.h> 中的数据结构体里面。

<include/linux/fs.h>有 3 个最重要的数据结构体

struct file_operations{ }：在内核的内部，由该结构体提供文件系统的入口点函数，即访问设备驱动的功能接口函数

```
struct inode{ };
struct file{ };
```

➤ file_operations 结构体的简便形式

```
struct file_operations fops={
read:      device_read,
write:     device_write,
open:      device_open,
release:   device_release}
```

➤ 设备驱动程序设计

- 1.设计驱动程序
- 2.编译和加载驱动程序

● 设计驱动程序

例2：一个简单的字符型设备驱动程序。

驱动程序模块	模块说明
Demo_read()	定义Demo设备的读取操作函数
Demo_write()	定义Demo设备的写入操作函数
Demo_ioctl()	定义Demo设备的ioctl函数
Demo_open()	定义Demo设备的打开操作函数
Demo_release()	定义Demo设备的关闭操作函数
Demo_ops={};	定义设备向系统注册的操作功能
Demo_Init()	定义系统的初始化模块
Demo_Exit()	定义系统的卸载模块
module_init(Demo_Init)	内核模块入口，完成模块初始化
module_exit(Demo_Exit)	卸载时的函数入口，完成模块卸载

```
/***** demo_driver 驱动程序 demo_drv.c*****/
#include<linux/kernel.h>
#include<linux/init.h>
```

```

#include<linux/fs.h>
#include<linux/module.h>
#define Demo_MAJOR 98
#define VERSION "Demo_Driver "
static void showversion(void)
{
    printk("*****\n");
    printk("\t%s\t\n",VERSION);
    printk("*****\n\n");}
/*Demo 设备的读操作接口函数*/
ssize_t Demo_read(struct file * file, char *buf, size_t count, loff_t *f_pos)
{
    printk("Demo_read[--kernel--]\n");
    return count;}

/*Demo 设备的写操作接口函数*/
ssize_t Demo_write(struct file* file, const char *buf, size_t count, loff_t *f_pos)
{
    printk("Demo_write[--kernel--]\n");
    return count;}
/*Demo 设备的 ioctl 接口函数*/
long Demo_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    printk("Demo_ioctl[--kernel--]\n");
    return 0;}

/*Demo 设备的打开设备接口参数*/
int Demo_open(struct inode *inode, struct file *file)
{
    printk("Demo_open[--kernel--]\n");
    return 0;}
/*Demo 设备的关闭接口函数*/
int Demo_release(struct inode*inode, struct file*file)
{
    printk("Demo_realse[--kernel--]\n");
    return 0;}

/*Demo 设备向系统注册操作功能,声明设备操作的功能接口函数*/
struct file_operations Demo_ops={
    open:      Demo_open,
    read:      Demo_read,
    write:     Demo_write,
    unlocked_ioctl: Demo_ioctl,
    release:   Demo_release,};
/*系统初始化*/
static int __init Demo_Init()
{
    int ret = -1;
    ret = register_chrdev(Demo_MAJOR,"demo_drv", &Demo_ops);
    showversion();
}

```

```

    if(ret < 0){
        printk("Demo_module failed with %d\n[--kernel--]", ret);
        return ret;    }
    else{
        printk("Demo_driver register success!!![--kernel--]\n"); }
    return ret;}
/*系统卸载*/
static void __exit Demo_Exit ()
{
    printk("Demo_driver unregister success!!![--kernel--]\n");
    unregister_chrdev(Demo_MAJOR,"demo_drv");
}

/*驱动程序版本及 GPL 协议证书信息*/
MODULE_DESCRIPTION("Simple int driver module");
MODULE_LICENSE("GPL");

/*内核模块入口,相当于 main()函数,完成模块初始化*/
module_init(Demo_Init);
/*卸载时调用的函数入口,完成模块卸载*/
module_exit(Demo_Exit);

```

● 编译和加载驱动程序

1、创建设备进入点——mknod

使用 **mknod** 命令在文件系统中创建设备进入点(设备文件)。 由于在设备驱动程序中已经定义其主设备号为 98，次设备号没有定义，故取默认值 0。

举例：

```
[root@BC root]# mknod /dev/demo_drv c 98 0
```

2、检查设备进入点是否创建成功——ls

```
[root@BC root]# ls -l /dev |grep demo_drv
```

3、编写 Makefile 文件

```

obj-m := demo_drv.o
KERNELDIR := /lib/modules/3.2.14/build
PWD := $(shell pwd)
modules:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
modules_install:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules_install
clean:
    rm -f *.ko ~core.depend

```

4、加载驱动程序——insmod

使用 **insmod** 命令加载驱动程序

```
[root@BC root]# insmod demo_drv.ko
```

5、编写用户测试程序，运行

例 3：编写一个调用设备驱动程序功能接口的**用户程序**。

```

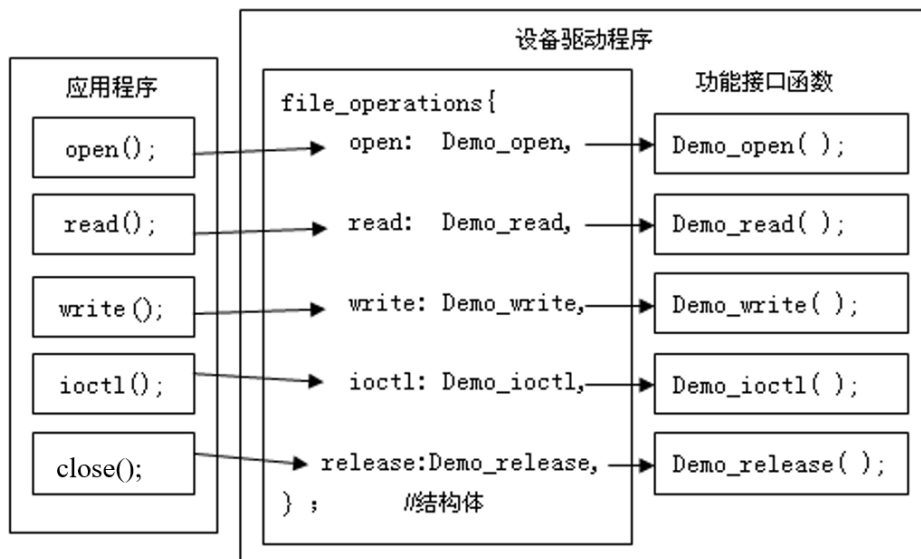
/***** 用户程序 test_driver.c *****/
#include<stdio.h>

```

```

#include<string.h>
#include<stdlib.h>
#include<fcntl.h>
#include<unistd.h>
#define DEVICE_NAME  "/dev/demo_drv"  //定义设备进入点
//main
int main()
{
    int fd;                                //声明文件描述符
    int ret;                                //记录关闭设备的返回值
    char *i;
    printf("\nstart Demo_driver test\n\n");
    fd = open(DEVICE_NAME,O_RDWR); //调用打开设备函数
    printf("fd=%d\n",fd);
    if(fd == -1)
    {
        printf("open device %s error\n",DEVICE_NAME);}
    else
    {
        read(fd,NULL,0); //调用驱动程序的读操作接口函数
        write(fd,NULL,0); //调用驱动程序的写操作接口函数
        ioctl(fd); //调用驱动程序的 ioctl 接口函数
        ret=close(fd); //若 ret 的值为 0，表示设备成功关闭
        printf("ret = %d\n",ret);
        printf("close Demo_drive test\n");}
    return 0;
}

```



在宿主机上

使用 gcc 编译，生成可执行文件 test_demo_drv

gcc test_driver.c -o test_demo_drv

运行

./test_demo_drv

在开发板上

使用 arm-linux-gcc 编译，生成可执行文件 test_demo_drv

arm-linux-gcc test_driver.c -o test_demo_drv

运行

```
./test_demo_drv
```

6、卸载驱动程序——rmmod

使用 **rmmod** 命令卸载驱动程序。

```
[root@BC root]# rmmod demo_drv
```

第 05 讲 嵌入式系统设备驱动程序 2

一、Linux 设备驱动程序代码分析

详情见上一讲

二、设备驱动程序模块参数传递

➤ 内核模块参数传递

应用程序参数传递

```
int main(int argc, char** argv)
```

```
./test 3 0 1
```

内核提供了一个简单框架，允许驱动程序声明参数，并且用户在系统启动或者模块装载时候为参数指定相应的值。

使用内核模块参数必须包含 linux/moduleparam.h 文件

➤ 模块参数定义方式

- ❑ `module_param(int_name, type, perm);`

- ❑ `module_param_named(ext_name, int_name, type, perm);`

- `int_name`: 当参数中没有 `ext_name` 的时候，此参数即是用户看到的参数名，又是模块内接受参数的变量
- `type`: 表示参数的数据类型，可以为 `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `charp`, `bool`, `invbool`
- `perm`: 指定了在 `sysfs` 中相应文件的访问权限。(一般不考虑)
- `ext_name`: 用户看到的参数名，对外的参数名

- ❑ `module_param_string(int_name, string, len, perm);`

- ❑ `module_param_array(int_name, type, nump, perm);`

- ❑ `module_param_array_named(ext_name, int_name, type, nump, perm);`

- `nump`: 指向一个整数，其值表示有多少个参数存放在数组中
- `len`: 当参数是 `string` 的时候，表示字符串 `string` 数组的大小

- ❑ 使用宏 `MODULE_PARM_DESC()` 对定义的参数进行说明

```
#include <linux/module.h>
```

```
#include <linux/moduleparam.h>
```

```
#include <linux/kernel.h>
```

```
#define MAX_ARRAY 6
```

```
static int int_var = 0;
```

```
static const char *str_var = "default";
```

```
static int int_array[6];
```

```
int narr;
```

```
module_param(int_var, int, 0);
```

```

MODULE_PARM_DESC(int_var, "A integer variable");
module_param(str_var, charp, 0);
MODULE_PARM_DESC(str_var, "A string variable");
module_param_array(int_array, int, &narr, 0);
MODULE_PARM_DESC(int_array, "A integer array");
static int __init hello_init(void)
{
    int i;
    printk(KERN_ALERT "Hello, my LKM.\n");
    printk(KERN_ALERT "int_var %d.\n", int_var);
    printk(KERN_ALERT "str_var %s.\n", str_var);
    for(i = 0; i < narr; i++){
        printk("int_array[%d] = %d\n", i, int_array[i]);
    }
    return 0;
}
static void __exit hello_exit(void)
{
    printk(KERN_ALERT "Bye, my LKM.\n");
}
module_init(hello_init);
module_exit(hello_exit);

```

加载命令：

`insmod demo_drv.ko int_var=100 str_var=hello int_array=100,200`

三、 设备驱动程序调试技术

➤ printk

printk()和C库中的printf()在使用上最主要的区别就是 **printk()指定了日志级别**。

```

#define KERN_EMERG "<0>" /* 系统不可使用 */
#define KERN_ALERT "<1>" /* 需要立即采取行动 */
#define KERN_CRIT "<2>" /* 严重情况 */
#define KERN_ERR "<3>" /* 错误情况 */
#define KERN_WARNING "<4>" /* 警告情况 */
#define KERN_NOTICE "<5>" /* 正常情况, 但是值得注意 */
#define KERN_INFO "<6>" /* 信息型消息 */
#define KERN_DEBUG "<7>" /* 调试级别的信息 */

```

➤ printk()的日志级别

- ☐ KERN_EMERG /* 用于紧急事件消息, 一般是系统崩溃消息 */
- ☐ KERN_ALERT /* 用于需要立即采取行动的情况 */
- ☐ KERN_CRIT /* 临界状态, 通常涉及严重的硬件或软件操作失败 */
- ☐ KERN_ERR /* 用于报告错误状态 */
- ☐ KERN_WARNING /* 对可能出现问题的情况进行告警 */
- ☐ KERN_NOTICE /* 有必要进行提示的正常情况 */
- ☐ KERN_INFO /* 提示性信息 */
- ☐ KERN_DEBUG /* 用于调试级别的信息 */

未指定优先级的 **printk** 语句采用的**默认级别**是

DEFAULT_MESSAGE_LOGLEVEL, 默认为 **KERN_WARNING**

```
printk(KERN_EMERG "log_level:%s\n", KERN_EMERG);
```



```
printk(KERN_DEBUG "The printer is still on fire\n");
printk("Using DEFAULT_MESSAGE_LOGLEVEL\n")
```

变量 `console_loglevel` 的初始值为 `DEFAULT_MESSAGE_LOGLEVEL`，只有级别小于 `console_loglevel` 的打印才会被输出到终端控制台

可以通过 `dmesg` 或 `cat /proc/kmsg` 指令来查看内核调试输出

- 可以通过修改 `/proc/sys/kernel/printk` 文件来修改控制台的日志级别。该文件中的四个整数值分别是：当前的日志级别、未明确指定日志级别时的默认消息级别、最小允许的日志级别以及引导时的默认日志级别

```
xduser@Ubuntu :~$ cat /proc/sys/kernel/printk
```

```
4          4          1          7
```

当前的日志级别 未明确指定日志级别时的默认消息级别 最小允许的日志级别 引导时的默认日志级别

➤ proc 文件系统

定义：

`proc` 文件系统是 Linux 中的特殊文件系统，提供给用户一个可以了解内核内部工作过程的可读窗口，在运行时访问内核内部数据结构、改变内核设置的机制。

- 保存系统当前工作的特殊数据，但不存在于任何物理设备中
- 对其进行读写时，才根据系统中的相关信息即时生成；或映射到系统中的变量或数据结构；
- `proc` 被称为‘伪文件系统’；
- 其挂接目录点固定为 `/proc`；

作用：

`/proc` 的文件可以用于访问有关内核的状态、计算机的属性、正在运行的进程的状态等信息。

大部分 `/proc` 中的文件和目录提供系统物理环境最新的信息。

尽管 `/proc` 中的文件是虚拟的，但它们仍可以使用任何文件编辑器或像 `'more'`、`'less'` 或 `'cat'` 这样的程序来查看。

`proc` 文件系统可以被用于收集有用的关于系统和运行中的内核的信息。下面是一些重要的文件：

```
/proc/cpuinfo - CPU 的信息 (型号, 家族, 缓存大小等)
/proc/meminfo - 物理内存、交换空间等的信息
/proc/mounts - 已加载的文件系统的列表
/proc/devices - 可用设备的列表
/proc/filesystems - 被支持的文件系统
/proc/modules - 已加载的模块
/proc/version - 内核版本
/proc/cmdline - 系统启动时输入的内核命令行参数
```

```
cat /proc/cpuinfo    查看/proc 中的文件 cupinfo
ls /proc/            查看/proc 文件下有什么文件
```

可以通过 `proc` 文件系统与内核进行交互

`hostname` (查看文件 `hostname`)

`cat /proc/sys/kernel/hostname`(查看仍然得到 Ubuntu)

`echo WorkStation > /proc/sys/kernel/hostname` 写入

➤ proc 文件系统的编程接口

/proc 目录下的文件属于一种特殊的文件，**必须使用特点的方法创建与删除。**

proc 文件系统的编程接口比较好记，大部分函数是 VFS 函数名前面加上一个“**proc_**”

- 创建目录函数 `proc_mkdir()`、`proc_mkdir_data()`;
- 创建符号链接函数 `proc_symlink()`;
- 创建 proc 文件函数 `proc_create()`、`proc_create_data()`;
- 删除 proc 节点函数 `remove_proc_entry()`;
- **struct proc_dir_entry 是 proc 文件系统的重要结构体,定义在 fs/proc/internal.h 中。**

```
struct proc_dir_entry {
    unsigned int low_ino;
    umode_t mode;    // mode: 文件的类型和权限
    nlink_t nlink;    // nlink: 该文件的链接数
    kuid_t uid;
    kgid_t gid;
    loff_t size;
    const struct inode_operations *proc_iops;
    const struct file_operations *proc_fops;    // proc_fops: 读写操作函数
    struct proc_dir_entry *next, *parent, *subdir; // parent: 创建节点的父目录
    void *data;
    atomic_t count;
    atomic_t in_use;
    struct completion *pde_unload_completion;
    struct list_head pde_openers;
    spinlock_t pde_unload_lock;
    u8 namelen;
    char name[];    // name: 节点的名称，也就是该 proc 文件的名称
}
```

➤ 在/proc 下创建目录 `proc_mkdir ()`

```
struct proc_dir_entry *proc_mkdir(const char *name, struct proc_dir_entry *parent)
```

参数:

name:要创建的目录名称

parent: 父目录，如果为 NULL，表示直接在/proc 下面创建目录

➤ 在/proc 下创建目录 `proc_mkdir_data ()`

```
struct proc_dir_entry *proc_mkdir_data(const char *name, umode_t mode, struct proc_dir_entry
*parent, void *data)
```

参数:

name:要创建的目录名称

mode:指定要创建目录的权限

parent: 父目录，如果为 NULL，表示直接在/proc 下面创建目录

data:保存私有数据的指针，如不需要则设为 NULL。

(PDE(inode)->data: 获取 `proc_mkdir_data` 传入的私有数据)

- **`proc_mkdir` 和 `proc_mkdir_data` 的区别在于他不能保存私有数据指针。**

➤ 创建 proc 虚拟文件系统文件 `proc_create ()`

```
struct proc_dir_entry *proc_create(const char *name, umode_t mode, struct proc_dir_entry *parent,
```

```
const struct file_operations *proc_fops)
```

参数:

name:要创建的文件名称

mode:指定要创建文件的权限

parent: 父目录, 如果为 NULL 表示直接在/proc 下面创建

proc_fops: proc 文件相关的操作函数

➤ **创建 proc 虚拟文件系统文件 proc_create_data ()**

```
struct proc_dir_entry *proc_create_data(const char *name, umode_t mode, struct proc_dir_entry *parent,  
const struct file_operations *proc_fops, void *data)
```

参数:

name:要创建的文件名称

mode:指定要创建文件的权限

parent: 父目录, 如果为 NULL 表示直接在/proc 下面创建

proc_fops: proc 文件相关的操作函数

data:保存私有数据的指针, 如不需要则设为 NULL。

- **proc_create 和 proc_create_data 的区别在于他不能保存私有数据指针。**

➤ **创建从 name 指向 dest 的符号链接 create_symlink()**

```
struct proc_dir_entry *proc_symlink(const char *name, struct proc_dir_entry *parent, const char *dest)
```

该函数在 parent 目录下创建一个名字为 name 的符号链接文件, 链接的目标是 dest。

参数:

name:要创建的符号链接名称

parent:要创建的符号链接文件所在的父目录, 如果为 NULL 表示直接在/proc 下面创建

dest:所要创建的符号链接指向的文件名。

- **该函数在用户空间等效为 ln -s dest name**

➤ **删除节点 (文件或者目录) remove_proc_entry()**

```
void remove_proc_entry ( const char *name, struct proc_dir_entry *parent)
```

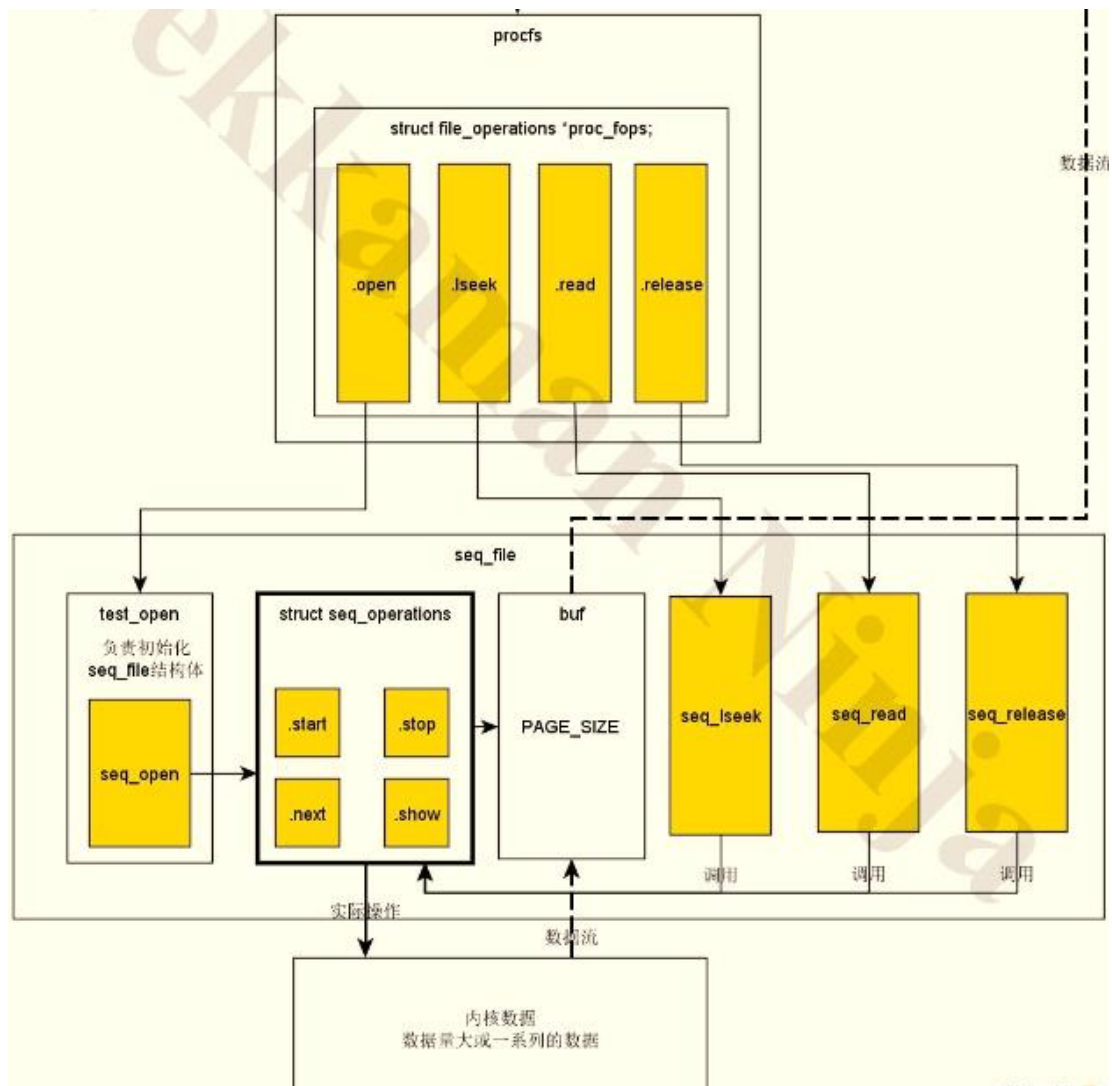
该函数将删除一个 proc 节点 (按文件名删除)。

参数:

name: 要删除的文件或者目录名。

parent: 符号所在的目录, 如果为 NULL, 表示在/proc 目录下

- 从内核中导出信息到用户空间有很多方法, 可以自己来实现 file_operations 的 read 函数或者 mmap 函数, 但是这种方法不够简单, 而且也会有一些限制, 比如一次 read 读取大于 1 页时, 驱动里就不得不去进行复杂的缓冲区管理, 容易出现 Bug。
- 内核黑客们对一些/proc 代码做了研究, 抽象出共性, 最终形成了 **seq_file** (Sequence file: 序列文件) 接口。
- 为了更简单和方便, 内核提供了 single_xxx 系列接口, 它是对 seq_file 的进一步封装。



➤ Seq_file 文件系统的编程接口

- Seq_file 必须实现四个操作函数：start(), next(), show(), stop()。

```
struct seq_operations {
    void * (*start) (struct seq_file *m, loff_t *pos);
    void (*stop) (struct seq_file *m, void *v);
    void * (*next) (struct seq_file *m, void *v, loff_t *pos);
    int (*show) (struct seq_file *m, void *v);};
```

- **start()**: 主要实现初始化工作，在遍历一个链接对象开始时调用。
- **stop()**: 当所有链接对象遍历结束时调用。主要完成一些清理工作。
- **next()**: 用来在遍历中寻找下一个链接对象。返回下一个链接对象或者 NULL (遍历结束)。
- **show()**: 对遍历对象进行操作的函数。主要是调用 seq_printf() 之类的函数，打印出这个对象节点的信息。

➤ 基于 Seq_file 接口的 Proc 文件系统实现

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
```

```
static struct mutex lock;
static struct list_head head;
struct my_data {
    struct list_head list;
    int value;
};
```

```
static void add_one(void) {
    struct my_data *data;

    mutex_lock(&lock);
    data = kmalloc(sizeof(struct my_data),
GFP_KERNEL);
    if (data != NULL)
        list_add(&data->list, &head);
    mutex_unlock(&lock);
}

static ssize_t _seq_write(struct file *file, const char
__user * buffer, size_t count, loff_t *ppos) {
    add_one();
    /* 此处使用copy_from_user可以从用户态传递数据到
内核态 */
    return count;
}
```

55

```
static void * _seq_start(struct seq_file *m, loff_t *pos) {
    mutex_lock(&lock);
    return seq_list_start(&head, *pos);
}
```

```
static void * _seq_next(struct seq_file *m, void *p, loff_t *pos) {
    return seq_list_next(p, &head, pos);
}
```

```
static void _seq_stop(struct seq_file *m, void *p) {
    mutex_unlock(&lock);
}
```

```
static int _seq_show(struct seq_file *m, void *p) {
    struct my_data *data = list_entry(p, struct my_data, list);
    seq_printf(m, "value: %d\n", data->value);
    return 0;
}
```

```
static struct seq_operations _seq_ops = {
    .start      = _seq_start,
    .next       = _seq_next,
    .stop       = _seq_stop,
    .show       = _seq_show
};

static int _seq_open(struct inode *inode, struct file *file) {
    return seq_open(file, &_seq_ops);
}

static struct file_operations _seq_fops = {
    .open        = _seq_open,
    .read         = seq_read,
    .write        = _seq_write,
    .llseek       = seq_lseek,
    .release      = seq_release
};
```

```

static int __init init(void) {
    struct proc_dir_entry *entry;
    struct my_data *data;

    mutex_init(&lock);
    INIT_LIST_HEAD(&head);
    add_one(); add_one(); add_one();

    entry = proc_create ("my_data",
S_IWUSR | S_IRUGO, NULL, &_seq_fops);
    if (entry == NULL) {
        return -ENOMEM;
    }
    return 0;
}

static void __exit fini(void)
{
    struct my_data *data;
    remove_proc_entry("my_data",
NULL);
    while (!list_empty(&head)) {
        data = list_entry((&head)-
>next, struct my_data, list);
        list_del(&data->list);
        kfree(data);
    }
}

module_init(init);
module_exit(fini);

```

➤ 基于 Single_xxxx 接口的 Proc 文件系统实现

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/types.h>
#include <linux/slab.h>
#include <linux/fs.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/mm.h>

MODULE_LICENSE("GPL");

typedef struct {
    int data1;
    int data2;
}DataType;

static DataType data[2];
static struct proc_dir_entry *procdir;

static int test_proc_show(struct seq_file *m,
void *v)
{
    DataType* pData = (DataType*)m->private;
    if(pData != NULL)
    {
        seq_printf(m, "%d----%d\n",
            pData->data1, pData->data2);
    }
    return 0;
}

static int test_proc_open(struct inode *inode, struct file *file)
{
    return single_open(file, test_proc_show, PDE(inode)->data);
}

static const struct file_operations dl_file_ops = {
    .owner          = THIS_MODULE,
    .open           = test_proc_open,
    .read           = seq_read,
    .llseek         = seq_lseek,
    .release        = single_release,
};

```

```

static int __init test_module_init(void) { /* create /proc/test */
    printk("[test]: module init\n");

    procdir = proc_mkdir("test", NULL);
    if (!procdir)
        return 0;

    data[0].data1=1;
    data[0].data2=2;
    proc_create_data("proc_test1", 0644, procdir, &dl_file_ops, &data[0]);

    data[1].data1=3;
    data[1].data2=4;
    proc_create_data("proc_test2", 0644, procdir, &dl_file_ops, &data[1]);
    return 0;
}

static void __exit test_module_exit(void)
{
    printk("[test]: module exit\n");
    remove_proc_entry("proc_test1", procdir);
    remove_proc_entry("proc_test2", procdir);
    remove_proc_entry("test", NULL);
}

module_init(test_module_init);
module_exit(test_module_exit);

```